Escola Politécnica da USP
Pós-Graduação em Sistemas Eletrônicos

PSI5000 – Vida Artificial em Ambientes Virtuais

Roger Tang

Sensoriamento, Redes Neurais Artificiais, Reconhecimento de Padrões, Classificação. Uma abordagem prática

Abstract. *This article describes how to use neural nets for recognition and classification of standards and as they can be used as sensory. Also it detailed how to define and to create a neural net in the practical aspect. To the end it presents an application that the user allows to create and to test its own net neural using the backpropagation technique.*

Resumo. *Este artigo descreve como empregar redes neurais para reconhecimento e classificação de padrões e como podem ser utilizadas como sensores. Também descreve detalhadamente como definir e criar uma rede neural no aspecto prático. Ao final apresenta uma aplicação que permite ao usuário criar e testar sua própria rede neural utilizando a técnica de retropropagação.*

1 Introdução

As Redes Neurais Artificiais também conhecidas como RNA´s, estão fundamentadas nos estudos sobre a estrutura do cérebro humano para tentar emular sua forma inteligente de processar informação. Alguns estudos da neurofisiologia consideram que a riqueza computacional do cérebro humano está associada ao grande número de neurônios, interconectados por uma rede complexa de sinapses.
A Rede Neural Artificial é um modelo matemático que imita o funcionamento do neurônio biológico e suas conexões. Basicamente cada neurônio recebe um conjunto de entradas representadas pelos dendritos e gera uma única saída representada pelo axônio. O núcleo da célula neural efetua a "soma" das entradas gerando uma saída. Cada conexão que liga as entradas com o núcleo possui um "peso" que está representado através das conexões sinápticas.

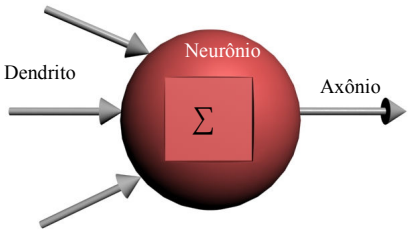

Figura 1 - Esquema básico de um Neurônio Artificial

As conexões sinápticas, doravante denominadas pesos, serão representadas graficamente por setas e algebricamente pela letra **w** (weight em inglês). As entradas de

um neurônio serão representadas algebricamente pela letra **x** e podem originar tanto da entrada inicial da rede quanto das saídas de outros neurônios. Analogamente ao neurônio biológico as entradas iniciais poderiam, por exemplo, vir diretamente de um nervo óptico ou qualquer outro órgão sensorial primário e ligar-se a rede de neurônios do cérebro.

Resumidamente o núcleo neural efetua inicialmente uma soma das entradas multiplicada por seus pesos como está representado matematicamente a seguir:

$$\sum_{i}^{n} w_i x_i$$

Abaixo está representada esquematicamente a soma de um único neurônio.

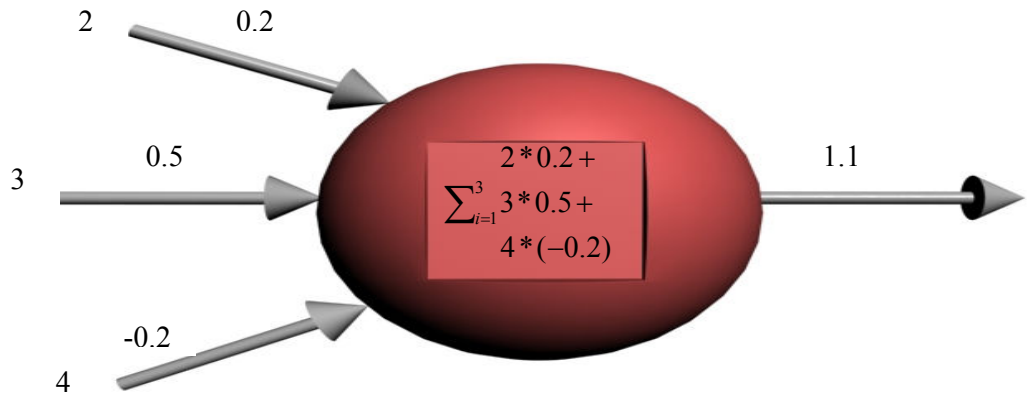

Figura 2 - Soma de um neurônio

A saída produzida pelo neurônio nesse caso seria 1.1, porém antes desse valor ser utilizado como entrada de outro neurônio é necessário primeiramente que passe por um processo de normalização.

A normalização da saída de um neurônio é um processo que faz com que todo resultado obtido esteja compreendido entre dois valores discretos, pois de outra forma, o resultado obtido poderia variar em todo campo dos números reais. Além disso, todo o universo de valores possíveis de serem gerados ficam compreendidos em uma faixa de valores que, por exemplo, pode ser de 0 a 1, de -1 a +1, ou outra faixa qualquer conforme a função adotada. Com a normalização é possível definir um ponto entre as faixas de valores que represente o **potencial de ativação** do neurônio.

O potencial de ativação do neurônio é o valor a partir do qual o neurônio irá ativar sua saída. Isso não significa que valores abaixo do potencial de ativação não produzem nenhuma saída, apenas que, analogamente ao neurônio biológico, o potencial de ativação serve como delimitador através do qual o neurônio é ativado.

Por exemplo: Se definirmos que a faixa de normalização está entre os valores -1 e 1 e que o potencial de ativação é 0.5, podemos dizer que o neurônio será ativado pelos valores acima de 0.5 como representados graficamente a seguir.

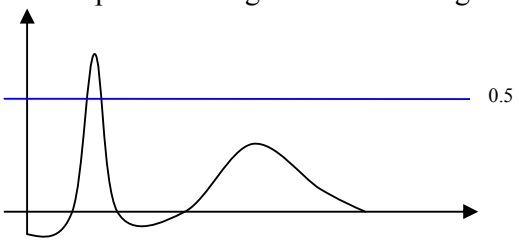

Figura 3 - Potencial de Ativação do neurônio

Para trabalharmos com o potencial de ativação, é necessário definirmos uma função de normalização, ou também chamada de função de ativação para o núcleo do neurônio. Existem diversas funções possíveis de serem utilizadas, porém a função deve conter algumas características importantes: Uma destas características é sempre mapear para um limiar inferior os valores negativos e para um limiar superior os valores positivos. Embora a 'função degrau' seja uma forte candidata, seus valores próximos a zero ficam sem um valor bem definido, por essa razão, uma função muito utilizada é a sigmoid que trabalha em uma faixa de -1 a +1 e os valores próximos a zero são mapeados em uma curva. Essa curvatura da sigmóide próxima ao valor zero, pode ser configurada alterando-se o valor β da função. A fórmula da sigmoid está representada a seguir:

$$f(x) = \frac{1}{1+\varepsilon^{-\beta x}}$$

Onde: β representa a curvatura da sigmoid.

Ao alterar este valor, a curvatura também é alterada.

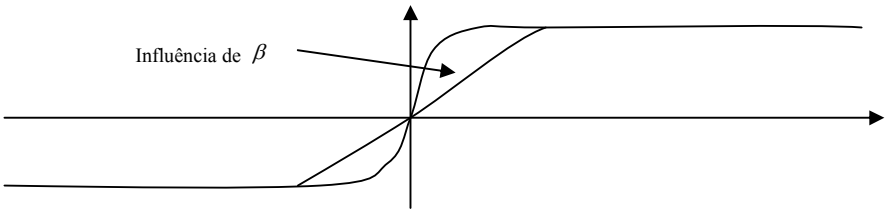

Figura 4 - Curva Sigmoid

Normalmente utiliza-se $\beta = 1$. Dessa forma, a saída do neurônio do nosso exemplo anterior ficaria:

$$f(1.1) = \frac{1}{1+\varepsilon^{-1(1.1)}} = 0.75026010559511769$$

Nesse caso em particular as entradas do exemplo "ativam" o neurônio. Isto se observa pois a saída 0.7 é maior do que 0.5 (valor definido anteriormente como sendo o potencial de ativação do neurônio). Essa característica dá à rede neural a possibilidade ser utilizada para **classificar padrões**.

Um caso particular de neurônio é o neurônio de McCulloch em que as entradas e a saídas são binárias.

2 Topologia da Rede

Uma rede neural possui diversas camadas. Cada camada de neurônio liga-se aos neurônios da camada seguinte e assim sucessivamente até que os neurônios da última camada produzam o resultado final. A cada interação de propagação da rede efetua-se a soma dos valores de entrada multiplicados por seu peso e em seguida normaliza o resultado como foi apresentado anteriormente. Esse processo é feito para todos os neurônios camada a camada.

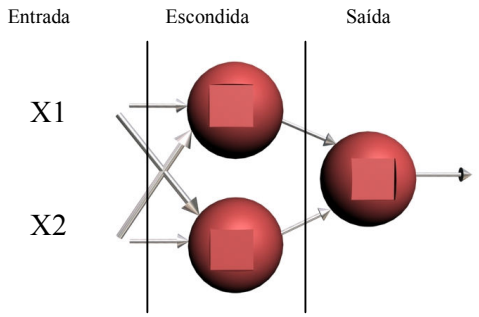

Figura 5 – Topologia de camadas de uma Rede Neural

Existe um motivo pelo qual uma rede neural possui diversas camadas, isto se dá devido a natureza do problema ao qual se quer resolver. Por exemplo, vamos analisar o problema da função AND. A tabela da verdade da função está representada a seguir:

Entrada 1	Entrada 2	Saída
0	0	0
0	1	0
1	0	0
1	1	1

Tabela 1 - Tabela da verdade da função AND

Essa função é dita linearmente separável, pois graficamente é possível definirmos uma reta (representada em verde) que separa as respostas, os resultados 0's dos 1's. As respostas 0´s estão representadas em azul e são os pontos (0,0),(0,1),(1,0) e as respostas 1´s estão representadas em vermelho - ponto (1,1).

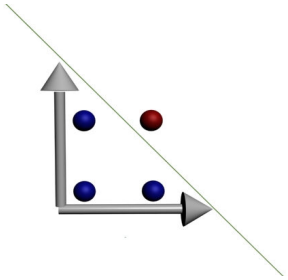

Figura 6 – Discriminador linear da função AND

Para solucionarmos o problema da função AND necessitamos de apenas duas camadas de neurônios: a camada de entrada e a de saída, e de apenas um neurônio, pois podemos definir um discriminador linear que separa o domínio das respostas em duas partes como está esquematizado pela linha verde.

Para o exemplo acima podemos definir a seguinte rede e seus respectivos pesos:

X1	W1	X2	W2	Saída	Sigmoid(saída)
0	1	0	1	(0*1)+(0*1)=0	0.5
0	1	1	1	(0*1)+(1*1)=1	0.7310
1	1	0	1	(1*1)+(0*1)=1	0.7310
1	1	1	1	(1*1)+(1*1)=2	0.8807

Tabela 2 - Tabela do neurônio da função AND

Graficamente temos:

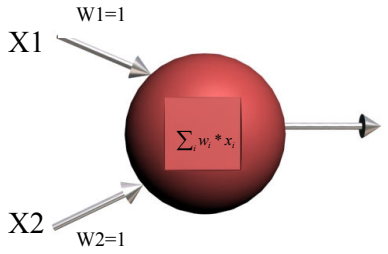

Figura 7 - Rede de duas camadas

Nesse caso o potencial de ativação do neurônio é o valor acima de 0.8807.
Como a função AND pode ser dividida em duas partes, ou seja, respostas menores e maiores do que 0.8807 dizemos que o problema é linearmente separável através deste discriminador linear.

Outro exemplo clássico é o da função XOR onde a tabela da verdade está definida a seguir:

Entrada 1	Entrada 2	Saída
0	0	0
0	1	1
1	0	1
1	1	0

Tabela 3 - Tabela da verdade da função XOR

No caso da função XOR, não é possível criar um discriminador linear no plano que separe os resultados 0´s dos 1´s em duas partes, contudo, é possível traçar um separador linear na terceira dimensão como mostra o gráfico abaixo:

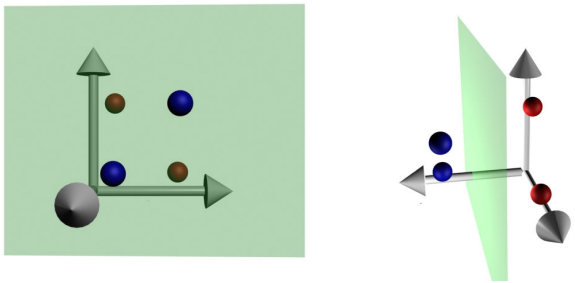

Figura 8 – Separação. Discriminador linear no plano

O separador está representado esquematicamente no plano verde. Dessa forma, acrescentando mais uma dimensão, é possível separar as respostas 0's (azuis) das 1's (vermelhas). Essa dimensão representa uma nova camada na rede neural.
Na nova rede tridimensional, as primeiras duas entradas são as mesmas, contudo o terceiro resultado é calculado em função dos primeiros. É um AND das duas primeiras entradas. Ajustando-se o peso da terceira camada temos a saída desejada.

X1	W1	X2	W2	X3= X1 AND X2	W3	Saída
0	1	0	1	0	-2	(0*1)+(0*1)+(0*-2)=0
0	1	1	1	0	-2	(0*1)+(1*1)+(0*-2)=1
1	1	0	1	0	-2	(1*1)+(0*1)+(0*-2)=1
1	1	1	1	1	-2	(1*1)+(1*1)+(1*-2)=0

Tabela 4 - Tabela do neurônio da função XOR

Observação: No exemplo acima a saída não foi normalizada por razões didáticas, porém é importante salientar que se a função sigmoid for utilizada, os pesos e as conexões também deverão ser alterados.

Finalmente podemos concluir que ao se acrescentar novas camadas às redes neurais podemos transformar qualquer problema em linearmente separável.

3 Treinamento de uma Rede

Para os problemas que são linearmente separáveis e possuem três dimensões podemos definir uma rede neural com três camadas.
Uma vez definida a topologia de uma rede neural é necessário definir seus pesos (w) de forma a se obter na saída os resultados desejados de uma determinada entrada. Os pesos de uma rede são obtidos através de um processo denominado de treinamento. Esse processo consiste em definir inicialmente pesos aleatórios, e através de diversas interações corrigem-se tais pesos de forma a obter os pesos que gerem os resultados desejados.
Uma forma de treinamento bastante utilizada é chamada de Backpropagation. Como o nome em inglês diz, o mecanismo desse treinamento consiste em propagar os valores desde a camada de entrada até a saída, calcular o erro de cada camada e depois *retropropagar* esses erros camada a camada de traz para frente. Esse processo é feito repetitivamente até que o erro global da rede atinja um valor aceitável.

A seguir será descrito um exemplo de cálculo de retropropagação para a função XOR.
O treinamento é feito da seguinte forma: Apresenta os valores (0,0) na entrada, propagam-se estes valores ao longo da rede e corrige os pesos a partir do valor desejado (0). Em seguida, apresenta os valores (0,1) com a resposta (1) e em seguida propagam-se os valores novamente. Finalmente repete-se este processo para os demais valores (1,0) com (1) e (1,1) com (0). Ao finalizar a correção de todos os casos de treinamento, inicia-se o processo novamente a partir do primeiro caso e assim sucessivamente até que as condições de parada sejam atendidas.

Caso	Entrada	Saída
1	0,0	0
2	0,1	1
3	1,0	1
4	1,1	0
1	...	...

Tabela 5 - Ciclos de Treinamento da função XOR

Em nosso primeiro caso iremos treinar as entradas 0 e 0 gerando a resposta 0, podemos facilmente concluir que este treinamento de nada irá valer, pois não importam quais pesos atribuirmos as saídas sempre serão zero. Em vista desse problema, é necessário inserirmos mais uma entrada em cada neurônio que será denominado de BIAS. Esta entrada terá sempre seu valor 1 para qualquer caso de treinamento. Isto faz com que uma saída diferente de zero seja produzida para as entradas (0,0).
A rede neural que irá resolver esse problema deverá ter duas camadas e está representada a seguir.

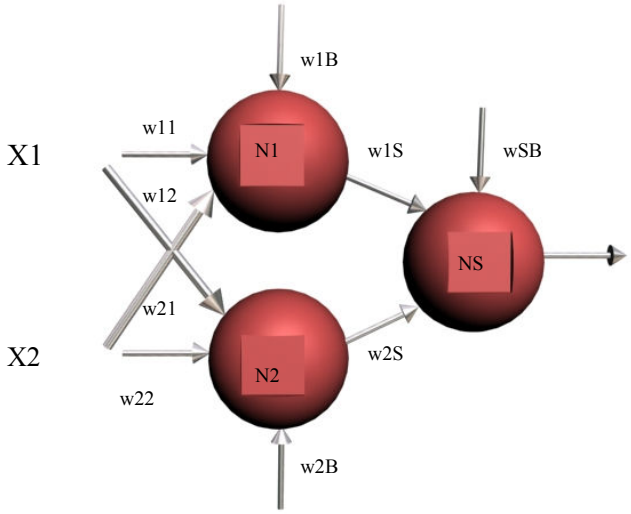

Figura 9 - Rede da função XOR

A notação dos pesos adotada acima está baseada no índice de origem e destino de cada neurônio. Exemplo w11 simboliza o peso da entrada X1 para o neurônio N1, w2S peso do neurônio N2 para o neurônio NS e assim sucessivamente.

Dessa forma, na primeira interação de propagação temos:

Cálculo de propagação da primeira camada
Cálculo de N1:

somaN1 = $\sum_{i}^{n} w_i x_i$ = (X1 * w11) + (X2 * w21) + (1 * w1B)

saídaN1 = $\frac{1}{1+\varepsilon^{-1*(\text{somaN1})}}$

Cálculo de N2:

somaN2 = $\sum_{i}^{n} w_i x_i$ = (X1 * w12) + (X2 * w22) + (1 * w2B)

saídaN2 = $\frac{1}{1+\varepsilon^{-1*(\text{somaN2})}}$

Cálculo de propagação da segunda camada
Cálculo de NS:

somaNS = $\sum_{i}^{n} w_i x_i$ = (saídaN1 * w1S) + (saídaN2 * w2S) + (1 *wSB)

saídaNS = $\frac{1}{1+\varepsilon^{-1*(\text{somaNS})}}$

Para utilizarmos exemplos numéricos vamos associar alguns valores aleatórios aos pesos como, por exemplo:

N1		N2		NS	
w11	-5,542640213	w21	3,66593471	w1S	-7,134121056
w12	-5,518389644	w22	3,655126122	w2S	-7,172954455
w1B	1,927722119	w2B	-5,68429643	wSB	3,531807947

Tabela 6 – Pesos aleatórios da função XOR

Teremos para o primeiro caso "0.0 gerando 0":

Cálculo de N1:

somaN1 = (X1 * w11) + (X2 * w21) + (1 * w1B) =
(0 * w11) + (0 * w21) + (1 * 1,927722119) = 1,927722119

saídaN1 = $\frac{1}{1+\varepsilon^{-1*(\text{somaN1})}}$ = 0,872997078478192

Cálculo de N2:

somaN2 = (X1 * w12) + (X2 * w22) + (1 * w2B) =
(0 * w12) + (0 * w22) + (1 * -5,68429643) = -5,68429643

saídaN2 = $\frac{1}{1+\varepsilon^{-1*(\text{somaN2})}}$ = 0,00338741024013824

Cálculo de NS:

SomaNS = (saídaN1 * w1S) + (saídaN2 * w2S) + (1 * wSB)
(0,872997078478192 * -7,134121056) +
(0,00338741024013824 * -7,172954455) +
(1 * 3,531807947) = -2,72055663177067

saídaNS = $\frac{1}{1+\varepsilon^{-1*(\text{somaNS})}}$ = 0,0617711985109306

Nesse ponto temos a saída de NS. Note que se não houvesse o BIAS a saída seria zero para esse caso de treinamento.
Agora iniciaremos o cálculo do erro para o caso da função sigmoid. Este erro é dado pelo mínimo do gradiente da função e é calculado da seguinte forma:

Cálculo do Erro da segunda camada
ErroNS = saídaNS * (1 – saídaNS) * (SaídaEsperada – saídaNS)

Cálculo do Erro da primeira camada
ErroN1 = saídaN1 * (1– saídaN1) * ErroNS * w1S
ErroN2 = saídaN2 * (1– saídaN2) * ErroNS * w2S

Numéricamente temos:

Cálculo do Erro da segunda camada
ErroNS = saídaNS * (1 – saídaNS) * (SaídaEsperada – saídaNS)

Neurônio NS
ErroNS = 0,0617711985109306 * (1 - 0,0617711985109306) * (0 – 0,0617711985109306) =
ErroNS = -0,00357998177910395

Cálculo do Erro da primeira camada
Neurônio N1
ErroN1 = saídaN1 * (1 – saídaN1) * ErroNS * w1S
ErroN1 = 0,872997078478192 * (1 - 0,872997078478192) * -0,00357998177910395 * -7,134121056 =
ErroN1 = 0,00283170359643779

Neurônio N2
ErroN2 = saídaN2 * (1 – saídaN2) * ErroNS * w2S
ErroN2 = 0,00338741024013824 * (1 - 0,00338741024013824) * -0,00357998177910395 * -7,172954455 =
ErroN2 = 0,0000866908087761718

Antes de calcularmos os pesos para a próxima interação devemos definir a velocidade com o qual a função erro irá caminhar a cada passo de interação. Essa velocidade é denominada de **taxa de aprendizagem da rede** e serve para que os pesos não fiquem saltando de forma a não convergirem para uma solução. De outra forma também servem para impedir que o treinamento ande muito devagar. Normalmente utiliza-se a taxa como sendo 0.5. Com os erros de cada camada podemos recalcular os pesos para a nova interação. Desta forma o cálculo dos novos pesos fica:

PesoNovo = peso + (taxa de aprendizado * entrada * erro)

Camada 1 – Neurônio 1
w1B' = w1B + (taxa * B1 * erroN1)
w11' = w11 + (taxa * X1 * erroN1)
w12' = w12 + (taxa * X2 * erroN1)
Camada 1 – Neurônio 2
w2B' = w2B + (taxa * B2 * erroN2)
w21' = w21 + (taxa * X1 * erroN2)
w22' = w22 + (taxa * X2 * erroN2)
Camada 2 – Neurônio S
wSB' = wSB + (taxa * BS * erroNS)
w1S' = w1S + (taxa * N1 * erroNS)
w2S' = w2S + (taxa * N2 * erroNS)

Exemplo Numérico:

w1B = 1,927722119 + (0,5 * 1 * 0,00283170359643779) = 1,92913797079822
w11 = -5,542640213 + (0,5 * 0 * 0,00283170359643779) = -5,542640213
w12 = -5,518389644 + (0,5 * 0 * 0,00283170359643779) = -5,518389644

w2B = -5,68429643 + (0,5 * 1 * 0,0000866908087761718) = -5,68425308459561
w21 = 3,66593471 + (0,5 * 0 * 0,0000866908087761718) = 3,66593471
w22 = 3,655126122 + (0,5 * 0 * 0,0000866908087761718) = 3,655126122

wSB = 3,531807947 + (0,5 * 1 * -0,00357998177910395) = 3,53001795611045
w1S = -7,134121056 + (0,5 * 0,872997078478192 * -0,00357998177910395) = -7,13568371281708
w2S = -7,172954455 + (0,5 * 0,00338741024013824 * -0,00357998177910395) = -7,17296051843347

A nova tabela de pesos após a correção fica:

N1		N2		NS	
w11	-5,542640213	w21	3,66593471	w1S	-7,13568371281708
w12	-5,518389644	w22	3,655126122	w2S	-7,17296051843347
w1B	1,92913797079822	w2B	-5,68425308459561	wSB	3,53001795611045

Tabela 7 – Pesos após a primeira interação da função XOR

Note que w11, w12, w21 e w22 não se alteraram, pois as entradas eram 0, porém os pesos associados aos BIAS fazem com que o valor dos pesos mude; no cálculo do próximo caso (0,1) resultando em 1 estes pesos irão se alterar.

4 Erro global e condição de parada

Para que o processo interativo de treinamento atinja seu final é necessário definirmos uma condição de parada.
A condição de parada do algoritmo pode ser feita de duas formas: ou define-se o número de interações máximo de propagações, ou define-se um erro global da rede o qual se deseja atingir. Existem vantagens e desvantagens em cada abordagem.
Na abordagem do erro global, temos que esse é calculado através do erro quadrático médio. Somam-se todas as variações dos erros da camada de saída como apresentado a seguir:

$$\Delta Erro = \frac{(SaídaEsperada - saida)^2}{2}$$

Exemplo numérico:

$$\Delta Erro = \frac{(0 - 0{,}0617711985109306)^2}{2} = 0{,}0019078404827384$$

Se a variação do erro foi menor ou igual a um valor definido, por exemplo, 1e-8 o algoritmo pára.

Existe um ponto importante nesta abordagem de parada: a função erro pode possuir vários mínimos locais e apenas um mínimo global. Isto pode fazer com que o algoritmo nunca alcance um determinado valor mínimo.

Graficamente temos:

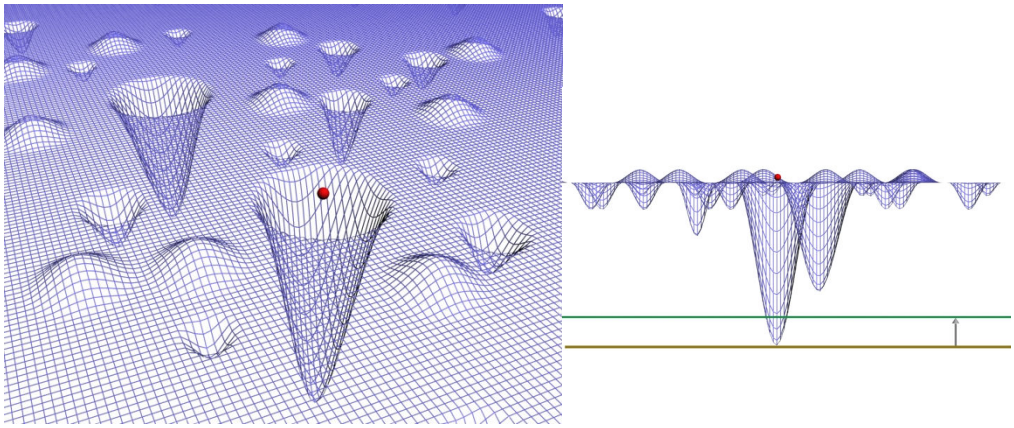

Figura 10 - Representação do erro da rede

A distância entre a linha verde e a linha amarela simboliza a variação do erro quadrático. Se o algoritmo cair em um mínimo local menor e tiver uma taxa de aprendizagem pequena, pode nunca atingir a variação de erro desejada. Nesse caso, o algoritmo não pára e devendo ser definida uma variação de erro maior.
Já a abordagem que se utiliza do número máximo de interações não possui esse problema, contudo nada garante que o treinamento chegue a uma solução adequada mesmo quando finalizadas todas as interações, pois o algoritmo também pode cair em um mínimo local.

5 Implementação de um Editor de Rede Neural

Esta sessão destina-se a descrever a implementação de um editor capaz de gerar, treinar e testar quaisquer redes neurais utilizando a técnica de retropropagação.
Para implementação de uma rede neural que pudesse ser genérica, simples e flexível, foi criada uma arquitetura de software que permite definir e utilizar a rede neural de forma independente. A arquitetura proposta desvincula o processo de criação ao de sua utilização, permitindo que o usuário da solução possa criar sua própria aplicação para testar novos casos em sua rede.
A solução é composta por um editor de rede neural e um componente COM, ambos implementados na plataforma Windows. O editor é uma aplicação win32 que serve para definir os parâmetros da rede, já o componente COM encapsula as funcionalidades de treinamento e de propagação da rede.

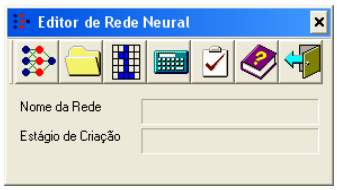

Figura 11 – Tela do Editor de Rede Neural

Para criar uma rede neural é preciso passar por algumas etapas que são:

Definição da Rede.

Definição dos padrões.

Treinamento da rede.

Teste dos padrões

6 Exemplo de criação de uma rede neural com Editor

A seguir será apresentado o processo de criação de uma rede com o editor de rede neural. Para este exemplo vamos utilizar a função XOR definida anteriormente.
Inicialmente o programa solicita ao usuário a definição dos parâmetros básicos da rede neural, o usuário entra com um identificador da rede (nome da rede neural), com o número de casos de testes (número de padrões) que serão utilizados no treinamento e com o número de linhas e colunas dos padrões de entrada e de saída. Com essas informações é possível definir a topologia da rede neural e a quantidade de neurônios. Importante salientar que não é necessário definir a camada escondida. A tela a seguir mostra a entrada dos parâmetros da rede:

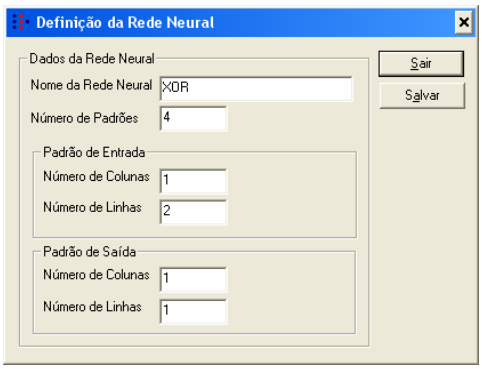

Figura 12 – Definição da rede neural

Ao finalizar essa etapa é gravado um arquivo chamado (XOR.rede) que guarda as informações inseridas nesse processo e a etapa de criação da rede. Nessa etapa o arquivo apenas possui as definições básicas e fica no estado de "**Rede definida**". Como mostrado a seguir.

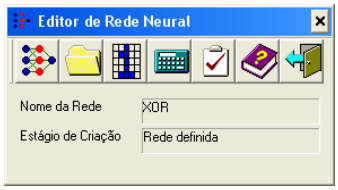

Figura 13 – Tela principal. Rede XOR no estado Rede definida

A próxima etapa é a inserção dos padrões que serão utilizados para o treinamento da rede. Neste exemplo utilizaremos quatro padrões a serem inseridos no editor para a função XOR como foi definido anteriormente que são:

Padrão de Entrada		Padrão de Saída
X1	X2	Saída
0	0	0
0	1	1
1	0	1
1	1	0

Tabela 8 – Tabela da verdade do XOR

Nessa etapa o usuário entra com os padrões que serão utilizados para o treinamento da rede neural conforme a tabela acima. A tela a seguir exemplifica esse processo:

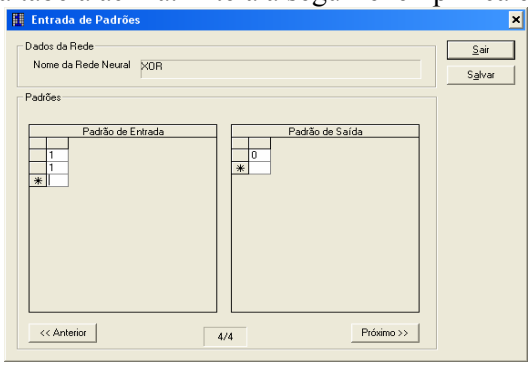

Figura 14 – Tela de entrada de padrões a serem treinados pela rede

Finalizada a entrada dos padrões que serão utilizados para o treinamento da rede neural, o arquivo (XOR.rede) fica no estado de "**Padrão definido**" como apresentado a seguir:

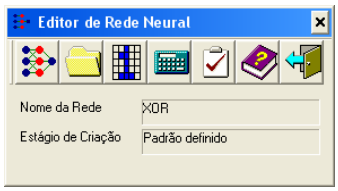

Figura 15 – Tela principal. Rede XOR no estado Padrão definido

Uma vez no estado Padrão Definido, a rede pode ser treinada. O usuário então deverá selecionar o tipo de treinamento. É possível treinar a rede de duas formas: pelo número máximo de interações ou pelo erro quadrático médio. Na tela a seguir o usuário pode escolher o tipo de treinamento e iniciar o processo de treinamento.

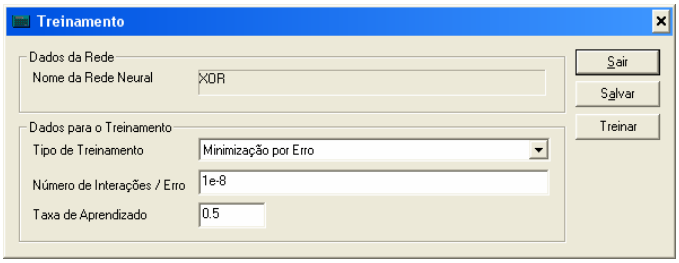

Figura 16 – Tela de treinamento da rede neural

Ao selecionar o botão treinar, o processo de treinamento inicia e a tela a seguir é apresentada enquanto o treinamento não finalizar.

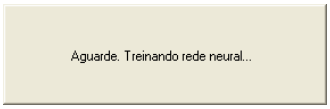

Figura 17 – Tela informando ao usuário que o editor está processando

Finalizado o processo de treinamento o usuário é informado que o arquivo de peso foi gerado. Esse arquivo contém não apenas a definição da rede neural, mas os pesos já calculados.

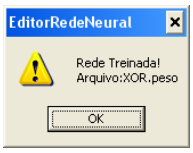

Figura 18 – Tela informando ao usuário que o treinamento finalizou

Quando for finalizado o processo de treinamento o arquivo (XOR.rede) passa a ficar no estágio de "**Treinamento definido**"

Figura 19 – Tela informando o estado do arquivo encontra-se em Treinamento

A partir deste ponto o arquivo (XOR.rede) não mais é utilizado a não ser que o usuário queira retomar algum estágio anterior e redefinir a rede. Neste instante, o arquivo (XOR.peso) gerado pelo processo de treinamento é utilizado para efetuar os teste dos padrões.
A última etapa do processo é o teste dos padrões. A tela a seguir exemplifica a entrada de um novo padrão e sua resposta obtida ao selecionar Testar Padrão.

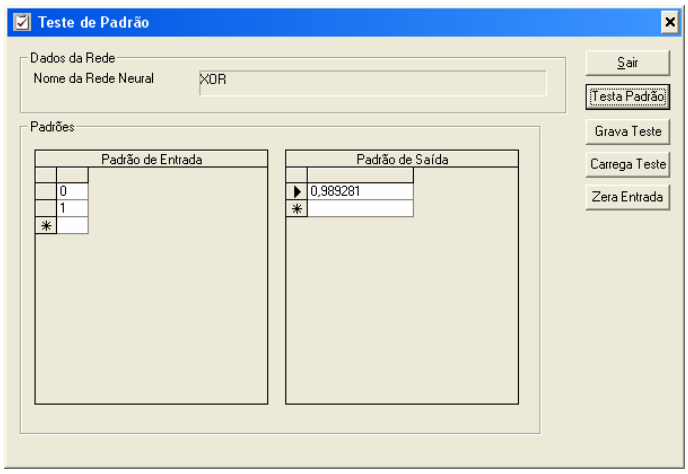

Figura 20 – Tela de teste de padrão

Analisando-se o exemplo acima, é possível notar que a saída da rede não é exatamente 1. Porém a saída é muito próxima de 1 e atinge o potencial de ativação do neurônio. O limiar do potencial de ativação deve ser definido conforme a aplicação, porém para esse caso podemos dizer que a saída atingiu seu limiar.
Se o potencial de ativação não for atingido, a rede pode ter convergido para um mínimo local menor do que se esperava e o processo de treinamento deve ser repetido.
Na tela de teste de padrão, também é possível salvar o padrão para um arquivo (.teste). Esse arquivo é utilizado para carregar facilmente um determinado teste específico. Como o usuário pode definir qualquer tamanho de padrão, podem existir padrões que possuam muitos elementos de entrada, fazendo com que esta função seja bastante útil.

7 Interface COM do Editor

O componente possui duas interfaces: uma de treinamento e outra de propagação. Ambas as interfaces são utilizadas pelo editor de rede, porém existe uma interface que pode ser utilizada por aplicações externas para testar novos padrões. Essa interface é utilizada para propagação de novos casos e está definida a seguir.

```
Propaga (string nomeArquivoPeso, string padrao, string padraosaida)
```

Onde:

nomeArquivoPeso: É o nome do arquivo gerado pelo editor de rede neural sem extensão.

Padrão: É uma string com o padrão a ser testado

padraosaida: É uma string com o padrão de saída

O formato do padrão de entrada e de saída são os mesmos e estão definidos como sendo os valores serializados linha a linha separados por ponto e vírgula. No caso do XOR podemos definir um exemplo de conteúdo do arquivo como sendo: 1.000000;0.000000.

8 Exemplo de aplicação de Reconhecimento de padrão numérico utilizando a arquitetura.

A seguir será apresentada uma aplicação externa que utiliza o componente COM para testar novos padrões gerados pelo Editor de Rede neural.
Esta rede destina-se a reconhecer números de 0 a 9; possui 35 neurônios de entrada dispostos em uma matriz de 5 colunas e 7 linhas e possui 10 neurônios de saída dispostos em um vetor de 1 coluna e 10 linhas. Cada padrão numérico foi treinado com uma das saídas em 1 e as demais em 0. A aplicação utiliza a rede definida pelo editor e sua interface está apresentada a seguir:

Figura 21 – Tela da aplicação externa de reconhecimento de dígito

Essa aplicação basicamente possui cinco funções que são:

- “Marca Todas” onde é possível marcar todas as células com o valor 1
- “Limpa Tudo” onde todas as células são marcadas com o valor 0
- “Salva” onde o usuário pode salvar um determinado padrão que foi inserido
- “Carrega” onde um padrão que foi salvo pode ser carregado novamente
- “Verifica” onde é possível verificar um determinado padrão.

A chamada do método da função COM é bastante simples. Um fragmento de código que efetua a verificação codificado na linguagem “visual basic” está apresentado a seguir:

```
Dim m_ProcessadorRede As New REDENEURALLib.ProcessadorRede

Private Sub cmdVerifica_Click()
Dim sPadrao As String
Dim sPadraoSaida As String
Dim nSaida As Integer
    sPadrao = MontaString
    On Error GoTo Erro
    m_ProcessadorRede.Propaga App.Path + "\Reconhecedor Dígito", sPadrao, sPadraoSaida
    nSaida = TrataRetorno(sPadraoSaida)
    If nSaida < 0 Then
        LblSaida = "X"
    Else
        LblSaida = nSaida
    End If
    Exit Sub
Erro:
    MsgBox Err.Description
End Sub
```

A função MontaString serializa a matriz em um string separando os elementos por ponto e vírgula(;) e a função TrataRetorno verifica qual é o maior valor retornado da rede. Se todos os valores forem inferiores a 0.6 retorna X que representa que o padrão não atingiu o limiar de ativação.
Essa aplicação **classifica o padrão** de entrada em uma das saídas possíveis desta forma, alem de funcionar como um reconhecedor de padrões funciona como classificador de padrões.

A seguir está representado esquematicamente todo o processo:

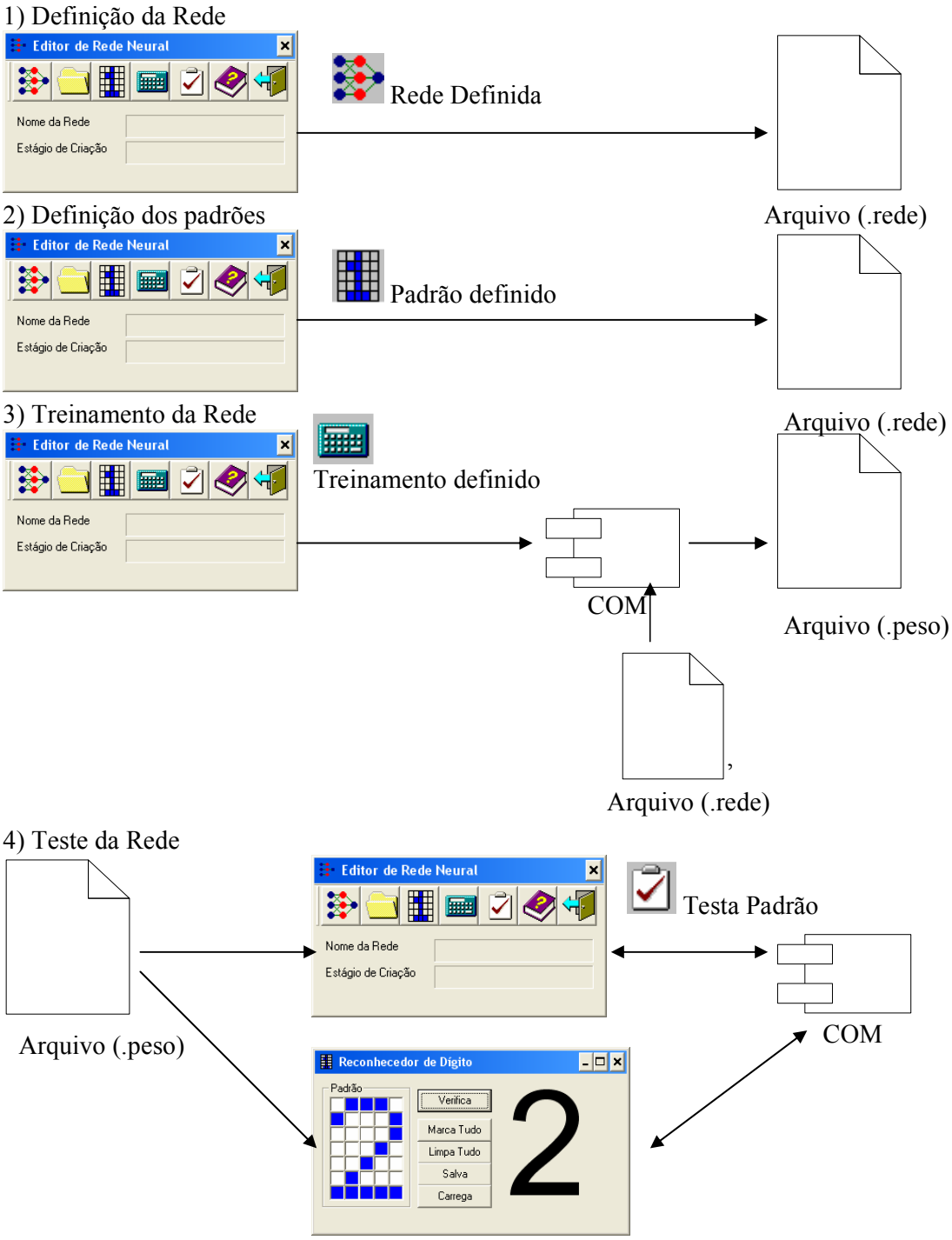

Figura 22 – Fluxo de utilização do Editor de Rede Neural

9 Outras Aplicações de Reconhecimento de Padrão

9.1 Sensoriamento

Uma rede neural pode ser utilizada como sensor. O exemplo acima poderia ser adaptado para reconhecimento de imagens. Dessa forma a rede associada a um dispositivo de entrada como uma câmera poderia reconhecer algumas formas e posteriormente transferir para outro módulo de processamento e decisão. Também é possível criar uma rede que tivesse como entrada um som e identificasse neste som alguns vocábulos e semelhantemente transferisse estes vocábulos a um outro módulo de processamento.

9.2 Classificação

A rede acima classifica os padrões de entrada em um dos dez tipos de saída, ou seja, um número parecido com um 1 é classificado como sendo 1. Uma outra aplicação que poderia ser utilizada como classificador seria, por exemplo, a obtenção de diversas características de um indivíduo para definição de seu perfil profissional. Inicialmente treina-se a rede com as características pessoais marcantes de cada profissional. Depois o candidato preenche o questionário e o resultado é um perfil do profissional. Interresante notar neste caso que mais de um padrão se apontado como saída da rede pode e deve ser observado.

10 Conclusão

Redes neurais mostram-se bastante eficientes para classificar e reconhecer padrões além de serem úteis no sensoriamento. Uma rede neural pode ser vista como uma função que mapeia valores de um domínio para uma imagem, contudo essa função é construída a partir de exemplos e não matematicamente. O processo de treinamento serve para obterem-se os pesos de uma rede que resolve um determinado problema e assemelha-se aos processos de cálculo numérico.

No caso da função XOR, criar um algoritmo que implemente a tabela da verdade da função seria bem mais eficiente. No entanto, a implementação utilizando a rede neural permite que valores não discretos e que não foram necessariamente previstos sejam utilizados como entrada. Por exemplo: XOR(0.2,0.8).

A rede também se torna bastante interessante quando é desejável reconhecer padrões mais complexos. No caso do reconhecedor de dígito definido anteriormente temos como entrada uma matriz de 5X7 que gera um total de 35 pontos. Se considerarmos que os pontos apenas assumem valores discretos 0 e 1 o total de combinações de entrada a ser classificado seria de $2^{35} = 34359738368$. Porém poderíamos considerar que ao invés de cada entrada apenas assumir apenas valores entre 0 e 1 poderia assumir todos os tons de cores entre o banco e o azul. Se considerarmos apenas 256 variações entre o branco e o azul teria $256^{35} = 1.94e84$ combinações. Dessa forma passa a ser bastante difícil criar um programa que teste todas estas combinações sem o auxílio de uma rede.

11 Referências Bibliográficas

KOVACS, Z L - *Redes Neurais Artificiais – Fundamentos e Aplicações* - Ed. da Física - 2002

HAYKIN, S S - *Redes Neurais - Principios e Prática* – Ed. Bookman Companhia - 2000

PLUNKETT, K., COHEN, G., JOHNSTON, R A - *Exploring Cognition, Damaged Brains and Neural Net* – Routledge - 2002

VEELENTURF, L P J - *Analysis and Applications of Artificial Neural Net* – Prentice Hall - 1995